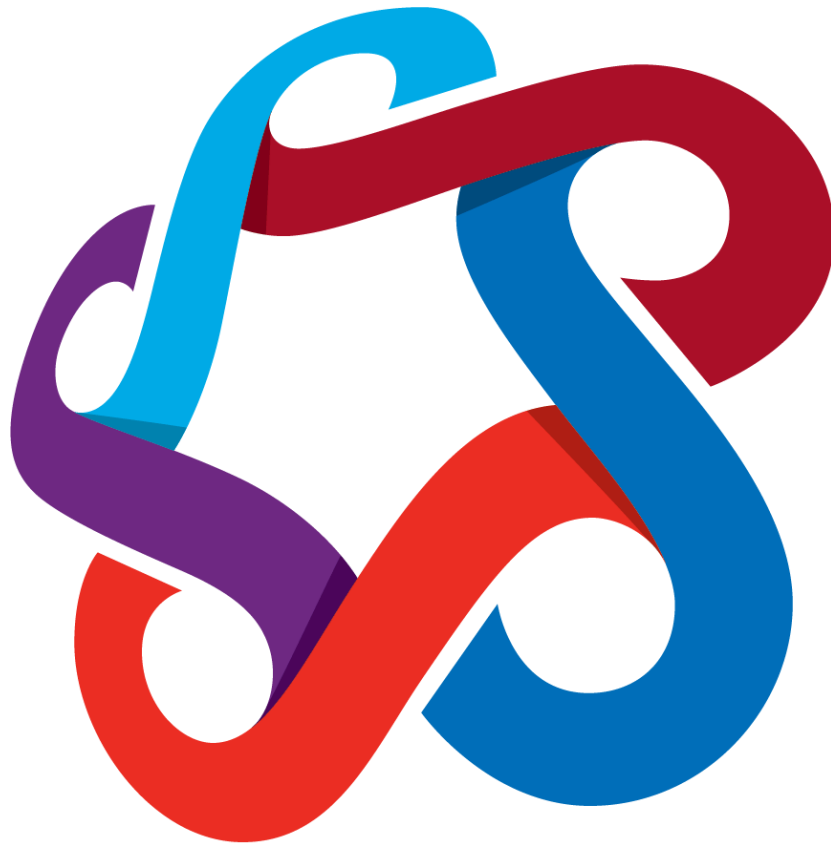


Building TLS Client-Server Applications with Sockets



SAIT

Abdulaziz Osman
09.26.2026

Client Interaction Step-by-Step Explanation

The interaction involves three main entities: the Client, the Server, and the underlying TLS certificate (generated locally)...

1. Certificate Generation (pre-req): Before any interaction, I run `generate_cert.py`. This script uses the cryptography library to generate an RSA 2048-bit private key (`server.key`) and a self-signed X.509 certificate (`server.crt`) with the common name "localhost".
2. TCP Connection Establishment: The client, `tls_chat_client.py` creates a standard TCP socket and attempts to connect to 127.0.0.1:8443. In the Wireshark capture, this is visible as the initial TCP three-way handshake can be seen on frames 1730-1732.

No.	Time	Source	Destination	Protocol	Length	Info
1730	49.402056900	127.0.0.1	127.0.0.1	TCP	56	37071 → 8443 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
1731	49.402114300	127.0.0.1	127.0.0.1	TCP	56	8443 → 37071 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
1732	49.402145100	127.0.0.1	127.0.0.1	TCP	44	37071 → 8443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
1733	49.402385800	127.0.0.1	127.0.0.1	TLSv1.3	561	Client Hello
1734	49.402404100	127.0.0.1	127.0.0.1	TCP	44	8443 → 37071 [ACK] Seq=1 Ack=518 Win=64768 Len=0
1735	49.403291700	127.0.0.1	127.0.0.1	TLSv1.3	1296	Server Hello, Change Cipher Spec, Application Data, Application Data, Application Data, Application Data
1736	49.403316900	127.0.0.1	127.0.0.1	TCP	44	37071 → 8443 [ACK] Seq=518 Ack=1253 Win=64256 Len=0
1737	49.404218500	127.0.0.1	127.0.0.1	TLSv1.3	124	Change Cipher Spec, Application Data
1738	49.404236900	127.0.0.1	127.0.0.1	TCP	44	8443 → 37071 [ACK] Seq=1253 Ack=598 Win=64768 Len=0
1739	49.404414800	127.0.0.1	127.0.0.1	TLSv1.3	299	Application Data
1740	49.404432000	127.0.0.1	127.0.0.1	TCP	44	37071 → 8443 [ACK] Seq=598 Ack=1508 Win=64000 Len=0
1741	49.404474500	127.0.0.1	127.0.0.1	TLSv1.3	299	Application Data
1742	49.404481800	127.0.0.1	127.0.0.1	TCP	44	37071 → 8443 [ACK] Seq=598 Ack=1763 Win=63744 Len=0
1743	49.404785600	127.0.0.1	127.0.0.1	TLSv1.3	80	Application Data
1744	49.404802000	127.0.0.1	127.0.0.1	TCP	44	37071 → 8443 [ACK] Seq=598 Ack=1799 Win=63488 Len=0
1902	54.840192900	127.0.0.1	127.0.0.1	TLSv1.3	69	Application Data
1903	54.840218400	127.0.0.1	127.0.0.1	TCP	44	8443 → 37071 [ACK] Seq=1799 Ack=623 Win=64768 Len=0
1904	54.840349200	127.0.0.1	127.0.0.1	TLSv1.3	116	Application Data
1905	54.840362100	127.0.0.1	127.0.0.1	TCP	44	37071 → 8443 [ACK] Seq=623 Ack=1871 Win=63488 Len=0
2694	76.686213900	127.0.0.1	127.0.0.1	TLSv1.3	85	Application Data
2695	76.686238200	127.0.0.1	127.0.0.1	TCP	44	8443 → 37071 [ACK] Seq=1871 Ack=664 Win=64768 Len=0
2914	84.427941800	127.0.0.1	127.0.0.1	TLSv1.3	93	Application Data
2915	84.427964400	127.0.0.1	127.0.0.1	TCP	44	37071 → 8443 [ACK] Seq=664 Ack=1920 Win=63488 Len=0

3. TLS Handshake & authentication: Once the TCP connection is established, the client wraps the raw socket using `context = ssl.create_default_context()`. It initiates a TLS handshake by sending a `Client Hello` (frame 1733). The server responds with a `Server Hello` and presents its certificate (`server.crt`) (1735). Because the certificate is self-signed, the client has been explicitly configured to bypass verification. They exchange parameters and switch to encrypted communication (Change Cipher Spec, Frame 1737).

No.	Time	Source	Destination	Protocol	Length	Info
1730	49.402056900	127.0.0.1	127.0.0.1	TCP	56	37071 → 8443 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
1731	49.402114300	127.0.0.1	127.0.0.1	TCP	56	8443 → 37071 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
1732	49.402145100	127.0.0.1	127.0.0.1	TCP	44	37071 → 8443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
1733	49.402385800	127.0.0.1	127.0.0.1	TLSv1.3	561	Client Hello
1734	49.402404100	127.0.0.1	127.0.0.1	TCP	44	8443 → 37071 [ACK] Seq=1 Ack=518 Win=64768 Len=0
1735	49.403291700	127.0.0.1	127.0.0.1	TLSv1.3	1296	Server Hello, Change Cipher Spec, Application Data, Application Data, Application Data, Application Data
1736	49.403316900	127.0.0.1	127.0.0.1	TCP	44	37071 → 8443 [ACK] Seq=518 Ack=1253 Win=64256 Len=0
1737	49.404218500	127.0.0.1	127.0.0.1	TLSv1.3	124	Change Cipher Spec, Application Data
1738	49.404236900	127.0.0.1	127.0.0.1	TCP	44	8443 → 37071 [ACK] Seq=1253 Ack=598 Win=64768 Len=0
1739	49.404414800	127.0.0.1	127.0.0.1	TLSv1.3	299	Application Data
1740	49.404432000	127.0.0.1	127.0.0.1	TCP	44	37071 → 8443 [ACK] Seq=598 Ack=1508 Win=64000 Len=0
1741	49.404474500	127.0.0.1	127.0.0.1	TLSv1.3	299	Application Data
1742	49.404481800	127.0.0.1	127.0.0.1	TCP	44	37071 → 8443 [ACK] Seq=598 Ack=1763 Win=63744 Len=0
1743	49.404785600	127.0.0.1	127.0.0.1	TLSv1.3	80	Application Data
1744	49.404802000	127.0.0.1	127.0.0.1	TCP	44	37071 → 8443 [ACK] Seq=598 Ack=1799 Win=63488 Len=0
1902	54.840192900	127.0.0.1	127.0.0.1	TLSv1.3	69	Application Data
1903	54.840218400	127.0.0.1	127.0.0.1	TCP	44	8443 → 37071 [ACK] Seq=1799 Ack=623 Win=64768 Len=0
1904	54.840349200	127.0.0.1	127.0.0.1	TLSv1.3	116	Application Data
1905	54.840362100	127.0.0.1	127.0.0.1	TCP	44	37071 → 8443 [ACK] Seq=623 Ack=1871 Win=63488 Len=0
2694	76.686213900	127.0.0.1	127.0.0.1	TLSv1.3	85	Application Data
2695	76.686238200	127.0.0.1	127.0.0.1	TCP	44	8443 → 37071 [ACK] Seq=1871 Ack=664 Win=64768 Len=0
2914	84.427941800	127.0.0.1	127.0.0.1	TLSv1.3	93	Application Data
2915	84.427964400	127.0.0.1	127.0.0.1	TCP	44	37071 → 8443 [ACK] Seq=664 Ack=1920 Win=63488 Len=0

tls_chat_client.py lines 21-25

```
def start_client():
    # Set up TLS Context (Disabling verification for local self-signed certs)
    context = ssl.create_default_context()
    context.check_hostname = False
    context.verify_mode = ssl.CERT_NONE
```

4. User Registration: Over the encrypted channel, the server sends an `ENTER_USERNAME` prompt. The client prompts the user (e.g., “Abdul”), sends the username, and the server registers the mapping `{Abdul: client_socket}` in its `clients` dictionary, replying with a `SUCCESS` message

```
(myenv) PS C:\Users\azolu\OneDrive\Documents\Code\School\Sem 5\Data Comm\Socket Programming> python .\tls_chat_client.py
Enter your username: Abdul
SUCCESS Connected to TLS Direct Messaging Server.

--- Instructions ---
Send direct messages using the format: recipient_username: message
Type 'exit' or 'quit' to close.
```

```
(myenv) PS C:\Users\azolu\OneDrive\Documents\Code\School\Sem 5\Data Comm\Socket Programming> python .\tls_chat_server.py
[*] TLS Direct Chat Server listening on 127.0.0.1:8443...
[+] User 'Abdul' connected from ('127.0.0.1', 37069)
[+] User 'Bob' connected from ('127.0.0.1', 37071)
```

5. Encrypted Messaging: Client 1 (Abdul) types a message formatted as `recipient_username: message` (e.g., “Abdul: Hello Abdul!”). The client encrypts this payload and sends it. The server decrypts it, parses the target, looks up Bob’s socket in the clients dictionary, formats the message, encrypts it, and routes it to client 2 (Bob). Bob’s client receives it, decrypts it, and prints it to the console.

```
(myenv) PS C:\Users\azolu\OneDrive\Documents\Code\School\Sem 5\Data Comm\Socket Programming> python .\tls_chat_client.py
Enter your username: Abdul
SUCCESS Connected to TLS Direct Messaging Server.

--- Instructions ---
Send direct messages using the format: recipient_username: message
Type 'exit' or 'quit' to close.

[Bob -> You]: Hello Abdul!

> Bob: TLS Works!
>
```

```
(myenv) PS C:\Users\azolu\OneDrive\Documents\Code\School\Sem 5\Data Comm\Socket Programming> python .\tls_chat_client.py
Enter your username: Bob
SUCCESS Connected to TLS Direct Messaging Server.

--- Instructions ---
Send direct messages using the format: recipient_username: message
Type 'exit' or 'quit' to close.

> Abdul: Hello Abdul!
[Abdul -> You]: TLS Works!
```

Summary and Answers to Questions

1. How do TCP and TLS differ in their socket creation and setup?

TCP socket creation involves instantiating a raw socket, binding to a port, and listening/accepting connections. There's no cryptographic involvement. TLS, on the other hand, is a two-step process; following the TCP socket creation, the raw socket is then wrapped using an SSL context.

In `tls_chat_server.py`, this involves creating a context with `ssl.Purpose.CLIENT_AUTH`, loading the certificate and private key, and wrapping the accepted connection.

`tls_chat_server.py` lines 68–71

```
def start_server():
    # Set up TLS Context
    context = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
    context.load_cert_chain(certfile="server.crt", keyfile="server.key")
```

On the client side, it involves creating a default context, modifying verification settings to accept the self-signed cert, and wrapping the raw socket before connecting.

2. How do they handle connection reliability, ordering and encryption?

Reliability and Ordering: TLS does not implement its own reliability or ordering mechanisms; it relies entirely on the underlying TCP layer. TCP handles packet retransmission, sequencing, and flow control.

Encryption: Plain TCP sends data in cleartext, as it has no native encryption; TLS provides confidentiality by encrypting payloads using symmetric keys established in the handshake.

3. What challenges did you face implementing TLS?

- Generating `server.crt` and `server.key` via standard openssl CLI. I resolved this by generating a python generation script using the cryptography library (`generate_cert.py`) to create the RSA key pairs programmatically.
- Capturing traffic targeted at 127.0.0.1 initially yielded empty trace logs because windows bypasses physical network interfaces for loopback sockets. In order to capture the packets, I had to select the **Adapter for loopback traffic capture**.

4. What are the trade-offs when choosing TLS over TCP?

Pros: TLS provides confidentiality against eavesdroppers, data integrity, and authentication (I disabled it for this implementation). Without it, any message could be read by anyone.

Cons: TLS introduces performance overhead — increased latency during setup as a result of the cryptographic key exchange (adding 1–2 RTTs). It also complicates network debugging, as tools like Wireshark can no longer read the plaintext application data without extra configuration.

5. Does TLS effectively address the security weaknesses you observed in TCP?

Yes, by applying TLS over TCP:

- Confidentiality is achieved: Packet inspection in Wireshark reveals only unreadable application data payloads.
- Integrity is enforced: Any modification of packets in transit breaks MAC validation, causing the receiving socket to reject the corrupted record.
- Server authentication is guaranteed: Clients verify that they are speaking to the true holder of `server.key` via public key signature verification during the initial handshake.

However, in this specific implementation, authentication is disabled.